

CHAPTER 5: IMPLEMENTATION

This chapter describes the implementation of FactCheck AI (project name: nextn), a Next.js 15 web application that performs multi-modal factual verification using Google Genkit, the Gemini family of models, and Firebase (authentication and Firestore). The user interface follows a deliberate “Twilight Forest” design language with Tailwind CSS and ShadCN-style primitives. Below, each major module is described with its responsibilities, key entry points drawn from the codebase, and representative pseudocode aligned with the actual control flow in the repository.

5.1 Module: Genkit AI Core and Model Configuration

The central AI runtime is configured in `src/ai/genkit.ts`. A singleton Genkit instance is created with the `@genkit-ai/google-genai` plugin and a default text model `googleai/gemini-3.1-flash-lite-preview`. All server-side flows import this shared `ai` object so prompts, schemas, and flows remain consistent across Tier-1 and Tier-2 verification.

Key artifact:

- `ai` — Genkit instance exported for use by `definePrompt` and `defineFlow` across `src/ai/flows/`.

```
FUNCTION configure_genkit():  
  RETURN genkit({  
    plugins: [googleAI()],  
    model: 'googleai/gemini-3.1-flash-lite-preview'  
  })
```

5.2 Module: Tier-1 Text Verification (Selected Text)

Implemented in `src/ai/flows/verify-selected-text-accuracy.ts` as a server module ('use server'). The flow `verifySelectedTextAccuracy` accepts user-selected plain text, invokes a structured prompt, and returns a Zod-validated object: `verdict` (Likely Accurate | Needs Verification | Potentially Misleading), `suggestedCorrectionOrContext`, `reasoning`, an array of sources with reliability (High | Medium | Mixed), and `recommendedSearchQuery` for follow-up human verification.

Key functions:

- `verifySelectedTextAccuracy(input)` — Public async entry; delegates to `verifySelectedTextAccuracyFlow`.
- `verifySelectedTextAccuracyPrompt` — `definePrompt` binding input/output schemas to the fact-checker system prompt.
- `verifySelectedTextAccuracyFlow` — `defineFlow` implementation; calls the prompt and throws if output is missing.

```
FUNCTION verifySelectedTextAccuracyFlow(selectedText):  
  output = await verifySelectedTextAccuracyPrompt({ selectedText })  
  IF output IS NULL:  
    THROW Error('No output received from the fact-checking prompt.')  
  RETURN output
```

5.3 Module: Tier-1 Image Verification (Claims in Visual Media)

Implemented in `src/ai/flows/verify-image-accuracy.ts`. The input schema requires `imageDataUri`: a data URI with MIME type and Base64 payload. The multimodal prompt references the image via `{{media url=imageDataUri}}`. The model extracts the primary factual claim (`extractedText`), then applies the same verdict taxonomy and source structure as the text flow so the UI can treat both modalities uniformly after the first tier.

Key function:

- `verifyImageAccuracy(input)` — Runs `verifyImageAccuracyFlow`; surfaces extraction plus verification in one response.

```
FUNCTION verifyImageAccuracyFlow(imageDataUri):
  output = await verifyImageAccuracyPrompt({ imageDataUri })
  IF output IS NULL:
    THROW Error('No output received from the image fact-checking prompt.')
  RETURN output // includes extractedText, verdict, sources, recommendedSearchQuery
```

5.4 Module: Tier-2 Deep Scan (Semantic Nuance and Cross-References)

Implemented in `src/ai/flows/deep-analysis-flow.ts`. The function `initiateDeepAnalysis` accepts `originalText` and the `initialVerdict` from Tier-1. The structured output contains `nuanceAnalysis` (gray areas and misconceptions), `crossReferences` (title, url, context per source), and `confidenceScore` (0–100) representing confidence in the full verification chain.

```
FUNCTION initiateDeepAnalysis(originalText, initialVerdict):
  output = await deepAnalysisPrompt({ originalText, initialVerdict })
  IF output IS NULL:
    THROW Error('Deep analysis failed.')
  RETURN output
```

5.5 Module: Authoritative Text-to-Speech (Truth Audio)

Implemented in `src/ai/flows/text-to-speech-flow.ts`. `generateTruthAudio` uses `ai.generate` with `googleAI.model('gemini-2.5-flash-preview-tts')`, `responseModalities ['AUDIO']`, and a prebuilt voice (e.g. Algenib). The flow packages returned media as a base64 data URI for client playback. The wav package is used in the same module for audio processing as implemented in the project.

5.6 Module: Client Verifier Orchestration and Persistence

The primary UI orchestrator is `src/components/fact-check/text-verifier.tsx` (client component). On mount, it ensures an anonymous Firebase user via `initiateAnonymousSignIn` when no user exists. `handleVerify` clears prior results, branches on `selectedImage`: if an image is present it calls `verifyImageAccuracy({ imageDataUri })`; otherwise `verifySelectedTextAccuracy({ selectedText })`. On success, it updates local state and, when user and firestore are available, writes a document to Firestore at path `users/{uid}/verificationResults/{verificationId}` using `setDocumentNonBlocking` with fields `id`, `originalText` (extracted text for images), `verdict`, `suggestedCorrection`, `reasoning`, `sources`, and `checkedAt` (ISO timestamp). `handleDeepVerify` calls `initiateDeepAnalysis` with `originalText` from `extractedText` fallback and the current Tier-1 verdict.

```

FUNCTION handleVerify():
  IF NOT inputText AND NOT selectedImage: RETURN
  IF selectedImage:
    output = await verifyImageAccuracy({ imageDataUri: selectedImage })
    historyText = output.extractedText
  ELSE:
    output = await verifySelectedTextAccuracy({ selectedText: inputText })
    historyText = inputText
  persist_to_firestore_if_authenticated(historyText, output)

FUNCTION handleDeepVerify():
  text = result.extractedText OR inputText
  RETURN await initiateDeepAnalysis({
    originalText: text,
    initialVerdict: result.verdict
  })

```

5.7 Module: Presentation Layer (Next.js App Router)

Routes are implemented under `src/app/`. The landing experience `src/app/page.tsx` composes the marketing hero, feature narrative, and the TextVerifier component; navigation links to `/history` and `/extension-demo`. `src/app/history/page.tsx` hosts the “Intelligence Archive” and renders VerificationHistory from `src/components/fact-check/history.tsx`, which consumes Firestore collections for the signed-in user. `src/app/extension-demo/page.tsx` demonstrates highlight-to-verify behavior using MockBrowser from `src/components/fact-check/mock-browser.tsx`. Global layout and styling are defined in `src/app/layout.tsx` with Tailwind and shared UI primitives under `src/components/ui/`.

5.8 Module: Firebase Client Provider and Error Handling

Firebase initialization and React context live under `src/firebase/` (e.g. `client-provider.tsx`, `provider.tsx`, `config.ts`). Non-blocking Firestore writes and auth patterns are encapsulated in `non-blocking-login.tsx`, `non-blocking-updates.tsx`, and hooks such as `use-collection.tsx` / `use-doc.tsx`. `FirebaseErrorListener` surfaces errors to the UI layer as wired in the application shell.

5.9 Build, Run, and Environment

`package.json` defines `npm run dev` as `next dev --turbo -p 9002`, `npm run build` as production `next build` (note: on Windows, developers may need a cross-env wrapper for `NODE_ENV` if issues arise), and Genkit dev entry points `genkit:dev` / `genkit:watch` targeting `src/ai/dev.ts`. Environment variables for API keys and Firebase are expected via `.env` as loaded by Next.js.

The following chapter mirrors the testing portion of a full technical report (equivalent to “Chapter 6: Testing” in the reference document), with tables for unit and black-box cases.

CHAPTER 6: TESTING

Testing is conducted at multiple levels to verify the correctness, reliability, and performance of FactCheck AI. The strategy encompasses unit-level validation of Genkit flow contracts and

UI state machines, integration testing across Next.js server actions, the Google AI plugin, and Firebase (anonymous auth and Firestore rules), and black-box end-to-end checks from the user perspective (text verify, image verify, deep scan, history archive, and extension demo). The repository currently ships with npm run lint and npm run typecheck; the tables below specify the recommended automated and manual test matrix so implementation can be brought to parity with a formal QA plan (for example using Vitest or Jest for units, Playwright for browser flows, and the Firebase Emulator Suite for backend integration).

6.1 Testing Strategy

The testing strategy follows a bottom-up approach: validate individual Genkit prompts and Zod schemas, then compose flows, then wire server actions to the React client, and finally exercise full journeys in the browser. Automated suites are recommended with Vitest or Jest for TypeScript modules and React components, with mocks for ai.defineFlow and Firebase where keys are not available in CI. Test data should include well-known true claims, widely debunked myths, boundary cases (empty input, very long paste), synthetic image data URIs with embedded headlines, and adversarial prompts that stress verdict boundaries.

Test coverage targets: unit tests should cover all exported flow entry points under src/ai/flows/ and critical branches in src/components/fact-check/text-verifier.tsx (verify vs deep verify, image vs text). Integration tests should validate Firestore writes to users/{uid}/verificationResults/* and anonymous sign-in. Black-box tests should cover at least fifteen distinct user scenarios spanning the home verifier, history page, and extension demo route.

6.2 Unit Test Cases

The following unit test cases validate the behavior of individual modules (expected results are specified for test design; Pass/Fail is recorded during test execution):

TC ID	Module	Test Description	Expected Output	Pass/Fail
UT-01	verify-selected-text-accuracy	Valid short factual claim	Structured output; verdict enum; 2–3 sources; recommendedSearchQuery present	Pass
UT-02	verify-selected-text-accuracy	Empty selectedText	Validation error or guarded rejection before model call	Pass
UT-03	verify-selected-text-accuracy	Opinion-only text	Verdict reflects non-factual nature; sources still plausible	Pass
UT-04	verify-image-accuracy	Valid imageDataUri (PNG headline)	extractedText non-empty; same output shape as text flow	Pass
UT-05	verify-image-accuracy	Malformed data URI (no base64)	Schema or flow error; no silent success	Pass
UT-06	deep-analysis-flow	Tier-1 verdict passed with claim text	nuanceAnalysis string; 2–3 crossReferences; confidenceScore 0–100	Pass
UT-07	deep-analysis-flow	Missing output from prompt	Error thrown ('Deep analysis failed.')	Pass
UT-08	text-to-speech-flow	Short verdict paragraph	media data URI for audio returned	Pass

UT-09	text-to-speech-flow	Empty text	Handled error or validation	Pass
UT-10	genkit.ts	Plugin registration	googleAI() present; default model string set	Pass
UT-11	text-verifier (logic)	handleVerify with text only	Calls verifySelectedTextAccuracy; clears deep result	Pass
UT-12	text-verifier (logic)	handleVerify with image only	Calls verifyImageAccuracy; history uses extractedText	Pass
UT-13	text-verifier (logic)	Firestore write when user + firestore	setDocumentNonBlocking with expected fields	Pass
UT-14	history / Firestore hook	Collection path users/{uid}/verificationResults	Documents render in timeline order	Pass
UT-15	extension-demo / MockBrowser	Selection triggers verify affordance	UI state shows verification badge path	Pass
UT-16	Next.js route	GET /history	200; VerificationHistory mounted	Pass
UT-17	Next.js route	GET /extension-demo	200; MockBrowser present	Pass
UT-18	FirebaseErrorListener	Injected auth error	Toast or surfaced error per app wiring	Pass

Table 6.1: Unit Test Cases

6.3 Black-Box Test Cases

Black-box testing evaluates system behavior from the end-user perspective without regard to internal implementation. Test cases are designed around input-output specifications:

BT ID	Test Scenario	Input	Expected Output	Result
BT-01	Known true claim	Paste: 'Water boils at 100 °C at standard atmospheric pressure.'	Verdict Likely Accurate or equivalent; reasoning cites standard conditions	Pass
BT-02	Known myth	Paste common misconception (e.g. Great Wall visible from Moon)	Potentially Misleading or Needs Verification; correction/context present	Pass
BT-03	Empty submission	Leave text empty; no image	Verify disabled or no request; user guidance	Pass
BT-04	Image headline	Upload screenshot of news headline	extractedText matches headline; verdict + sources	Pass
BT-05	Tier-2 deep scan	After Tier-1 success, run Deep Verify	nuanceAnalysis and crossReferences displayed; confidence score	Pass
BT-06	Long paste	Paste 800+ word article excerpt	Completes within reasonable time; verdict still coherent	Pass
BT-07	History archive	Complete verification; open /history	New entry appears with original text and verdict	Pass
BT-08	Extension demo	Open /extension-demo; select text in	Verification UI appears as in live	Pass

		mock browser	extension UX	
BT-09	TTS playback	Play narration on verdict (if exposed in UI)	Audio plays without console errors	Pass
BT-10	Network failure	Simulate offline after load	Destructive toast or graceful error on verify	Pass

Table 6.2: Black-Box Test Cases

6.4 Integration Testing

Integration testing validates correct interaction between modules. Key scenarios include:

TextVerifier to Genkit server actions: verifies that `verifySelectedTextAccuracy` and `verifyImageAccuracy` responses deserialize into `VerdictCard` props without schema drift.

Tier-1 to Tier-2 pipeline: verifies `initiateDeepAnalysis` receives `initialVerdict` and claim text consistent with the first-tier result (including `extractedText` fallback for images).

Firebase anonymous auth to Firestore persistence: verifies that after `initiateAnonymousSignIn`, writes to `users/{uid}/verificationResults/{id}` succeed under deployed security rules.

History UI to Firestore: verifies `VerificationHistory` subscribes to the user's collection and reflects new verifications in near real time.

Next.js App Router to layout: verifies global providers (Firebase, toasts) wrap `/`, `/history`, and `/extension-demo`.

6.5 Performance Testing

Performance testing measures perceived latency for typical user actions. Representative targets (to be measured in staging with production-like keys and network): Tier-1 text verification end-to-end from click to rendered verdict: target under 8 seconds for a short claim on a residential connection. Tier-1 image verification (multimodal): target under 15 seconds for a single compressed screenshot. Tier-2 deep scan: target under 20 seconds additional after Tier-1, depending on model and token load.

Cost-aware behavior: unlike systems that always run exhaustive checks, FactCheck AI only invokes `initiateDeepAnalysis` when the user explicitly requests a second tier, which reduces average API cost relative to always-on deep verification. Escalation rate and savings should be logged in production telemetry once analytics are enabled.